NOVA IMS INFORMATION MANAGEMENT SCHOOL

# Preparation Test

*Computation I / Introduction To Programming*

Version I

**Read the following instructions carefully:**

1. Write your full name, student number and course at the end of this page in a clear and readable manner.
2. Anonymous Tests or tests where the student name is not readable will not be evaluated.
3. The mini test has a duration of 1 hour and 30 minutes. The ending time of the test will be written by the professor on the board before the test begins.
4. The score of each question varies according to the group to which it belongs.
5. If the professor cannot read the student's answer due to poor calligraphy or the choice of more than one answer occurred, the answer will be considered wrong. It is the student's responsibility to type in an understandable manner.
6. All answers must be written on the pages of this document. No other paper or document will be evaluated unless it is previously approved and signed by one of the professors.
7. No documents (books, notes, etc...) are allowed to be used during the exam (nor computers or phones).
8. No communication with other students is allowed during the test.
9. All phones must be turned off before the beginning of the test. Students are not allowed to have their phones in their possession even if turned off. Same applies for smart watches and other digital devices.
10. The breaking of any of the previously mentioned rules will cause for you to be expelled from the room, invalidating the mini-test and the necessary disciplinary measures will be applied.

Complete Name: _____

Student number: _____

Practical class: ☐ P1 ☐ P2 ☐ P3 ☐ P4 ☐ P5 ☐ P6 ☐ P7 ☐ ISPA

Exam room: ☐ A120 ☐ Annex 1 ☐ Annex 2 ☐ INE

# Group 1

X points total, Y points each question

1. Consider the following code. What will be printed on the console if the user inserts 7?

```
n = input("Gimme a number please: ")
print(f"{n * 2} and {int(n) * 2} and {n + str(2)} and {len(n * 3)}")
```

   - ◯ 77 and 77 and 72 and 3
   - ◯ 77 and 14 and 72 and 3
   - ◯ 77 and 14 and 72 and 21
   - ◯ 14 and 14 and 9 and 21
   - ◯ 77 and 14 and 7 2 and 3
   - ◯ 77 and 14 and 9 and 3
   - ◯ An error occurs
   - ◯ None of the alternatives

2. Consider the following variables. Which of the following expressions evaluates to `True`?

```
a = 7
b = 3
s = "to be or not to be"
```

   - ◯ `len(s.split()) == b * 2 and not len(s) < a * 2`
   - ◯ `s.replace("be", "is").split()[1] == "be"`
   - ◯ `len(s.replace("o", "")) > len(s) - b`
   - ◯ `len(s.replace("be", "")) < b * 4 < len(s)`
   - ◯ `len(s.split("be")) == b and a > len(s)`
   - ◯ `len(s.replace(" ", "")) == len(s) - b * 2`
   - ◯ An error occurs
   - ◯ None of the alternatives

3. Consider the following code. What will be printed on the console if the user inserts `C-137` (`a`), then `3` (`b`) and then `schwifty` (`c`)?

```
a = input("Which dimension are you from? ")
b = int(input("How many Mortys do you have? "))
lt = [a, b]
c = input("Time to get... ")
lt.append(c)

res = "Wubba"
for i in range(b):
    res = res + a[i]
for x in lt[-1]:
    if x in "aeiou":
        res = res + x.upper()
print(f"Result: {res} {len(lt)}")
```

- ◯ `Result: C-1I 3`
- ◯ `Result: WubbaC-1I 8`
- ◯ `Result: WubbaC-13I 3`
- ◯ `Result: WubbaC-1I 2`
- ◯ `Result: WubbaC-1IY 3`
- ◯ `Result: WubbaC-1I 3`
- ◯ An error occurs
- ◯ None of the alternatives

# Group 2

X points total, Y points each question

**NOTE:** Each question in this group shows a block of code and states what that code was supposed to do. There is an issue with each block of code: it does not do what was intended. Your task is to find the issue and fix it, as follows:

**A** State the issue briefly on the line after "What is the issue?".

**B** Rewrite each faulty line of code, corrected, on the empty line directly below it. Leave all the other empty lines blank.

**Example of a complete answer.** The answers are written in blue, and the letters in the margin show where each step was answered.

This block of code was supposed to print `yayyy yummy!` when the favourite fruit of the user is grapefruit, however it is capitalised.

**A** **What is the issue?** *print() returns nothing; () missing after lower*

---

Correctly identify and **fix the problematic line(s)** below.

```
1 fruit = print(input("Gimme your favourite fruit?"))
```

**B** *fruit = input("Gimme your favourite fruit?")*

```
2
3 if fruit.lower == "grapefruit":
```

**B** *if fruit.lower() == "grapefruit":*

```
4     print("yayyy yummy!")
```

The questions of this group start on the next page.

1. This block of code was supposed to ask the user three times whether it will be alright. It should count how many answers contain the word yes, in any capitalisation. For every other answer, it should replace each vowel (capital or not) with `XX` and add the length of the result to `size`. In the end, it should print `I'm so glad you said yes X times!`, where `X` is that count (or `I hate it when you say that...` if no answer contains yes), followed by `The other answers add up to S characters.`
   For instance, for the answers `Yes!`, `nope` and `yes yes`, it should print:
   ```
   I'm so glad you said yes 2 times!
   The other answers add up to 6 characters.
   ```
   **What is the issue?** _____

   Correctly identify and **fix the problematic line(s)** below.

```
1  count = 0
2  size = 0
3  for i in range(1, 3):
4      idk = input("Hey traffic lights, will it be alright? ")
5      if "YES" in idk.lower():
6          count = count + 1
7      else:
8          for v in "aeiou":
9              idk.lower().replace(v, "XX")
10         size = size + len(idk)
11
12 if count == 0:
13     print("I hate it when you say that...")
14 else:
15     print("I'm so glad you said yes {count} times!")
16 print(f"The other answers add up to {size} characters.")
```

2. This block of code was supposed to separate the names of the guests and add Michael to the list. It should then welcome each guest with their name in capital letters and count how many guests have names longer than three letters. Since Pam is one of the guests, it should print:

```
Welcome, JIM!
Welcome, PAM!
Welcome, DWIGHT!
Welcome, MICHAEL!
Pam is in! Last to arrive: Michael
2 guests have long names.
```

**What is the issue?** _____

Correctly identify and **fix the problematic line(s)** below.

```
1  guests = "Jim, Pam, Dwight"
2  names = guests.split(",")
3  names = names.append("Michael")
4  long = 0
5  for name in names:
6      print(f"Welcome, {name.upper}!")
7      if len(name) > 3:
8          long = long + 1
9
10 if "Pam" in names:
11     print(f"Pam is in! Last to arrive: {names[len(names)]}")
12 print(f"{long} guests have long names.")
```

## Group 3

X points total, Y points each question

**NOTE:** This group corresponds to direct questions. Please answer the questions within the space provided for the answer. The amount of space provided is just speculative. Do not feel inclined to add more just to fill in the space.

1. Several people are joining the Citadel at once, and their names arrive in a single string, separated by semicolons. Consider the following block of code, which creates a username for each of them. What needs to be written instead of `<A>`, `<B>`, `<C>`, `<D>` and `<E>` so that the program works as described below?

   - The string is split into a list with one person in each position (`<A>`).
   - The name of each person is split into its parts (`<B>`).
   - The username is the first letter of the first name followed by the last name without hyphens, all in lowercase and with at most 6 characters (`<C>`).
   - If that username is already taken (`<D>`), the character 2 is added at its end.
   - The username is then added to the list of taken usernames (`<E>`).

   For instance, Rick Sanchez-Smith gets the username `rsanch`, and the second Morty Smith gets `msmith2`. In the end, the program prints:
   `['rsanchez', 'ssmith', 'bsmith', 'msmith', 'rsanch', 'msmith2']`

```
line = "Beth Smith;Morty Smith;Rick Sanchez-Smith;Morty Smith"
taken = ["rsanchez", "ssmith"]

for p in <A>:
    parts = <B>
    username = <C>
    if <D>:
        username = username + "2"
    <E>

print(taken)
```

(`<A>`): _____

(`<B>`): _____

(`<C>`): _____

(`<D>`): _____

(`<E>`): _____

# Group 4

X points total, Y points each question

**NOTE:** This group corresponds to open questions. Make sure you present all necessary code when code is requested from you. Feel free to write notes and other things to show your thought process when you feel like it may be necessary.

You are helping your colleagues find the room for the first coding test. The program starts with the following lines of code, which are already written: for each room, `starts` and `ends` give the first and the last student number of that room, in the same order as `rooms`, and `counts` will keep how many students go to each room.

```
rooms = ["A120", "Annex I", "Annex II", "INE auditorium"]
starts = [20080048, 20261213, 20261394, 20261481]
ends = [20261212, 20261393, 20261480, 20262195]
counts = [0, 0, 0, 0]
```

Continue the program, answering each of the following questions below the previous one.

1. Write the code that does the following:

   a) Ask for the student numbers, all in one line and separated by commas (spaces may appear between them).

   b) For each number, print the room it belongs to, using the lists above. If the number is in no room, print a message asking the student to contact the academic services.

   c) Each time a student is placed in a room, add 1 to the position of that room in `counts`.

   Example of a correct run of the program (what the user inserts is in grey; your messages may be worded differently as long as they convey the intended message):

   ```
   Student numbers: 20100001, 20261300,20261450, 1, 20100002
   20100001: A120
   20261300: Annex I
   20261450: Annex II
   1: please contact the academic services
   20100002: A120
   ```

   Write your answer in the box on the next page.

2. Print how many students go to each room, and then print the busiest room (the one with the most students).

   Example of a correct run of the program, continuing the example of the previous question (your messages may be worded differently as long as they convey the intended message):

```
A120: 2 students
Annex I: 1 students
Annex II: 1 students
INE auditorium: 0 students
The busiest room is A120.
```

   Write your answer in the box below.

3. The test is at 14:00. Ask for the names of the students, all in one line, separated by semicolons and in the same order as their student numbers (spaces may appear between them). Then, print the time at which each student must arrive, in the format `h:mm` (e.g. `13:05`). Students must arrive 30 minutes before the test, with the following exceptions:

   - If the name starts with the letter "b" (regardless of capitalisation; you know who you are...), 90 minutes before.
   - Otherwise, if the name contains "pe" or ends with "a", 55 minutes before.
   - Otherwise, if the student number is odd, 40 minutes before.

   Example of a correct run of the program, continuing the example of the previous questions (what the user inserts is in grey; your messages may be worded differently as long as they convey the intended message):

   ```
   Names: Bruno; Pedro;Maria ;Tom; Joao
   Bruno should arrive at 12:30.
   Pedro should arrive at 13:05.
   Maria should arrive at 13:05.
   Tom should arrive at 13:20.
   Joao should arrive at 13:30.
   ```

   Write your answer in the box on the next page.

**** EXTRA PAGE ****

Good work! ☺